



DR B. DIOP

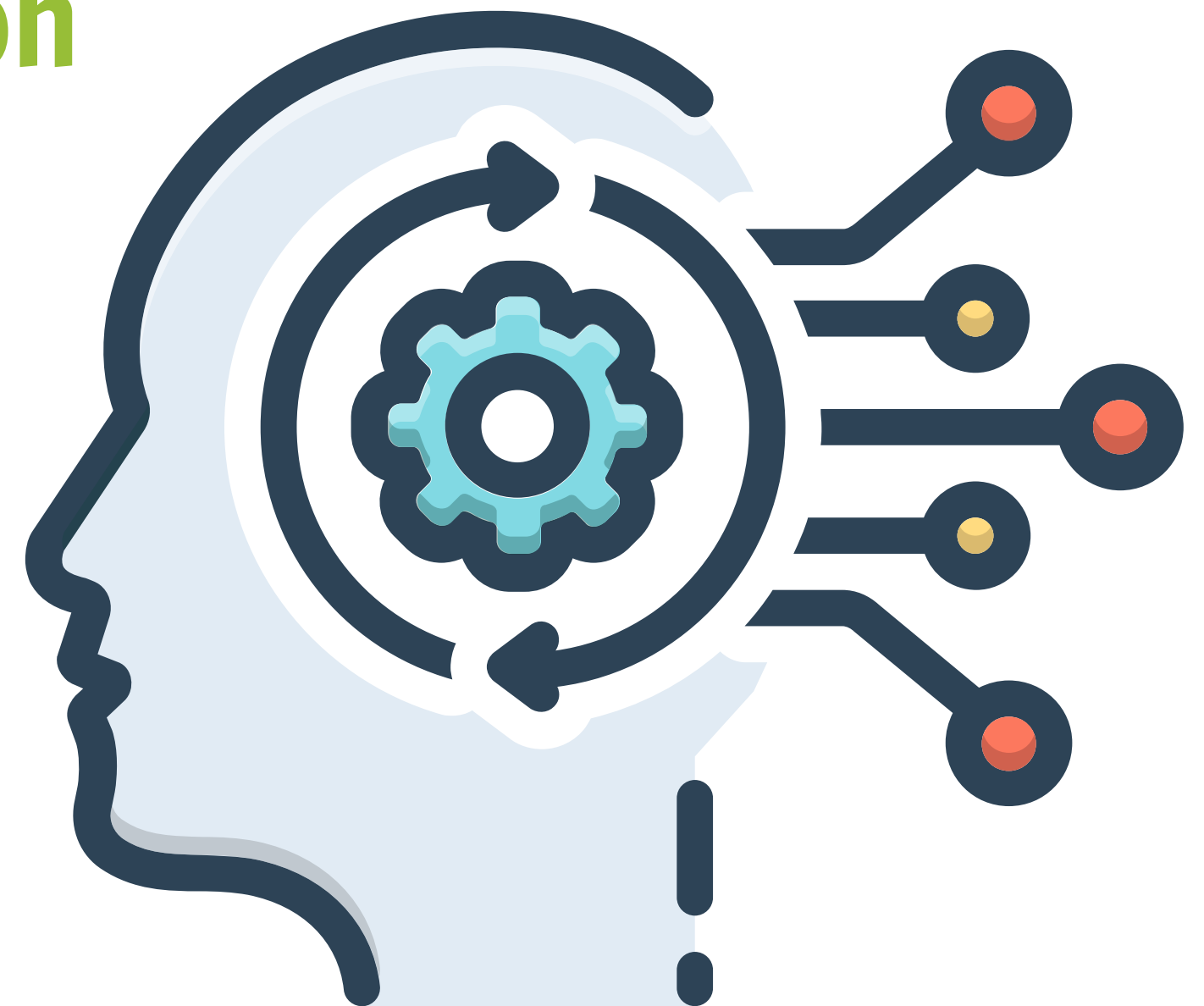
babacar.diop@ussein.edu.sn

Conteneurisation & orchestration

Intro to Docker

INFO – AGROTIC

Last update: oct. 2025



Outline

Syntaxe et commandes Docker

1. Introduction - Évolution de la syntaxe Docker
2. Gestion des images - Pull, build, push, suppression
3. Gestion des conteneurs - Création, démarrage, arrêt, suppression
4. Interaction avec les conteneurs - Exec, inspect, copie de fichiers
5. Mapping de ports - Exposition des services
6. Création d'images - Commit
7. Maintenance - Nettoyage avec prune
8. Exercices pratiques - Pour mettre en application
9. SOmmaire

Evolution de la syntaxe docker

Docker a révolutionné la manière dont nous déployons et gérons les applications.

Évolution de la syntaxe Docker

- Docker a évolué d'une approche de commandes simples vers un système orienté objet plus structuré.
- Bien que les anciennes commandes soient toujours supportées pour la compatibilité, la nouvelle syntaxe est recommandée.

Evolution de la syntaxe docker

Docker a évolué d'une approche de commandes simples vers un système orienté objet plus structuré.

Ancienne commande	Nouvelle commande	Description
<code>docker ps</code>	<code>docker container ls</code>	Lister les conteneurs
<code>docker run</code>	<code>docker container run</code>	Exécuter un conteneur
<code>docker rm</code>	<code>docker container rm</code>	Supprimer des conteneurs
<code>docker images</code>	<code>docker image ls</code>	Lister les images
<code>docker rmi</code>	<code>docker image rm</code>	Supprimer des images

Gestion des images : téléchargement

Télécharger une image (Pull)

La commande docker pull permet de télécharger une image depuis Docker Hub.

```
docker pull <nom_image>
```

Exemple : Téléchargement de l'image Redis

```
docker pull redis
```

Par défaut, Docker télécharge la version **latest**. Pour spécifier une version :

```
docker pull redis:6.2
```


Gestion des images : Lister les images

Lister les image (avec la commande **Pull**)

Pour afficher toutes les images disponibles localement, on utilise :

```
docker images
```

ou

```
docker image ls
```

Gestion des images : Supprimer des images

Supprimer une image (**rmi**)

Pour supprimer une image existante localement, on utilise :

```
docker rmi <image_ID_ou_nom>
```

```
docker image rm <image_ID_ou_nom>
```

Gestion des images : Construire des images

Construire une image (**build**)

Pour créer une image personnalisée localement, à travers un dockerfile :

```
docker build -t nom_image:tag .
```

Le point **.** indique le répertoire courant contenant le Dockerfile.

Gestion des images : Mettre son image sur Docker Hub

Pousser une image vers Docker Hub

Après avoir créé une image personnalisée, vous pouvez la partager :

```
docker login
```

```
docker push nom_image:tag
```

.Gestion des conteneurs

Gestion des conteneurs : démarrage d'un conteneur

Créer et démarrer un conteneur (Run)

- C'est la commande la plus fondamentale. Elle crée un nouveau conteneur à partir d'une image et le démarre :

```
docker container run <nom_image>
```

- Donner un nom au conteneur :

```
docker container run --name mon_conteneur <nom_image>
```

Options courantes :

- d : Mode détaché (exécution en arrière-plan)
- p : Mapping de ports
- e : Variables d'environnement

Gestion des conteneurs : Lister les conteneurs (PS)

Afficher les conteneurs en cours d'exécution

```
docker ps
```

ou

```
docker container ls
```

Options utiles :

- -a : Afficher tous les conteneurs (actifs et arrêtés)
- -l : Afficher le dernier conteneur créé
- -q : Afficher uniquement les IDs

Gestion des conteneurs : arrêter des conteneurs

Arrêter un ou des conteneurs en cours d'exécution

- Arrêter un seul conteneur :

```
docker container stop <container_ID>
```

ou

```
docker stop <container_ID>
```

- Arrêter plusieurs conteneurs :

```
docker stop conteneur1 conteneur2 conteneur3
```


Gestion des conteneurs : redémarrer un conteneur

Redémarrer un conteneur

- Opération utile pour résoudre des problèmes ou appliquer des changements :

```
docker restart <nom_conteneur_ou_ID>
```

Gestion des conteneurs : supprimer un conteneurs

Supprimer de conteneur (RM)

Supprimer un ou plusieurs conteneurs arrêtés :

```
docker rm <nom_conteneur_ou_ID>
```

ou

```
docker container rm <nom_conteneur_ou_ID>
```

Options importantes :

- -f : Forcer la suppression (même si le conteneur est en cours d'exécution)
- -v : Supprimer également les volumes associés

.Interagir avec des conteneurs

Interaction avec les Conteneurs

Exécuter des commandes dans un conteneur (Exec)

Permet d'exécuter des commandes dans un conteneur actif :

```
docker exec [options] <conteneur> <commande>
```

Exemple : Ouvrir un shell interactif

```
docker exec -it mon_conteneur bash
```

Options utiles :

- -i : Mode interactif (garde STDIN ouvert)
- -t : Alloue un pseudo-terminal
- -d : Exécution en arrière-plan

Afficher les processus d'un conteneur

Cette commande affiche les processus actifs dans le conteneur, similaire à la commande top sous Linux.

```
docker container top <nom_conteneur_ou_ID>
```

Inspecter un conteneur

Pour déboguer ou obtenir des informations détaillées :

```
docker inspect <nom_conteneur_ou_ID>
```

Retourne un fichier JSON avec toutes les configurations du conteneur.

Copier des fichiers

Transférer des fichiers entre l'hôte et le conteneur :

```
docker cp <source> <destination>
```

Exemples d'utilisation :

De l'hôte vers le conteneur

```
docker cp fichier.txt mon_conteneur:/chemin/
```

Du conteneur vers l'hôte

```
docker cp mon_conteneur:/chemin/fichier.txt ./
```

.Mapping de ports

Mapping de Ports

Pour accéder à un conteneur depuis l'extérieur, vous devez mapper les ports :

```
docker run -d -p <port_hôte>:<port_conteneur> <nom_image>
```

Exemple d'utilisation :

```
docker run -d -p 8080:80 nginx
```

Cela mappe le port 8080 de votre machine au port 80 du conteneur.

.Conteneur to image

Création d'Images à partir de Conteneurs

Commit

- Créer une nouvelle image à partir d'un conteneur modifié :

```
docker commit <nom_conteneur_ou_ID> nouvelle_image:tag
```

- Cette commande est utile pour sauvegarder l'état d'un conteneur après y avoir apporté des modifications.

.Maintenance et Nettoyage

Création d'Images à partir de Conteneurs

Commande Prune

- Nettoyer tous les objets Docker inutilisés en une seule commande :

```
docker system prune
```

- Cette commande supprime :
 - Tous les conteneurs arrêtés
 - Tous les réseaux non utilisés
 - Toutes les images pendantes (dangling images)
 - Tout le cache de build

Attention : Utilisez cette commande avec précaution en production !

Création d'Images à partir de Conteneurs

Sauvegarder une image

- Exporter une image sous forme de fichier :

```
docker save -o image.tar nom_image:tag
```

.Exercices pratiques

Exercices Pratiques

Exercice 1 : Démarrer un serveur web

1. Télécharger l'image nginx

```
docker pull nginx
```

2. Lancer le conteneur avec mapping de port

```
docker run -d -p 8080:80 --name mon_nginx nginx
```

3. Vérifier que le conteneur fonctionne

```
docker ps
```

4. Accéder à <http://localhost:8080> dans votre navigateur

Exercices Pratiques

Exercice 2 : Modifier et sauvegarder un conteneur

1. Lancer un conteneur Ubuntu

```
docker run -it --name mon_ubuntu ubuntu bash
```

2. Installer un package (dans le conteneur)

```
apt-get update && apt-get install -y curl
```

3. Quitter le conteneur (Ctrl+D)

4. Créer une nouvelle image

```
docker commit mon_ubuntu ubuntu-avec-curl:1.0
```

5. Vérifier la nouvelle image

```
docker images
```

.Résumé commandes Docker

Sommaire des commandes

Commande	Description
<code>docker pull</code>	Télécharger une image
<code>docker images</code>	Lister les images
<code>docker run</code>	Créer et démarrer un conteneur
<code>docker ps</code>	Lister les conteneurs actifs
<code>docker stop</code>	Arrêter un conteneur
<code>docker start</code>	Démarrer un conteneur
<code>docker rm</code>	Supprimer un conteneur
<code>docker rmi</code>	Supprimer une image
<code>docker exec</code>	Exécuter une commande dans un conteneur
<code>docker logs</code>	Afficher les logs d'un conteneur
<code>docker inspect</code>	Inspecter un conteneur ou une image
<code>docker system prune</code>	Nettoyer les ressources inutilisées

Exécuter des Commandes dans les Conteneurs Docker

Manipulation de Conteneurs

Introduction

Un conteneur Docker est un package logiciel léger, autonome et exécutable qui contient tout le nécessaire pour faire fonctionner une application : le code, l'environnement d'exécution, les outils système, les bibliothèques et les paramètres.

Les conteneurs Docker sont isolés les uns des autres et du système hôte, garantissant une exécution cohérente quelle que soit l'infrastructure sous-jacente.

Manipulation de Conteneurs

Objectifs du cours

- Exécuter des commandes interactives dans de nouveaux conteneurs
- Utiliser docker **exec** pour interagir avec des conteneurs en cours d'exécution
- Comprendre la différence entre **docker attach** et **docker exec**
- Automatiser l'exécution de commandes via **Dockerfile**
- Exécuter plusieurs commandes simultanément dans un conteneur

Interactif avec docker run : principe

Méthode 1 : Démarrer un Shell Interactif avec **docker run**

Principe

- Cette méthode vous permet de démarrer un nouveau conteneur et d'obtenir immédiatement une invite de commande (comme bash) à l'intérieur.
- C'est idéal pour explorer une nouvelle image ou configurer un environnement.
- **Syntaxe :**

```
docker run -it <nom_image> <shell>
```

Interactif avec docker run : syntaxe et utilisation

Méthode 1 : Démarrer un Shell Interactif avec **docker run**

- Syntaxe et détails des options

```
docker run -it <nom_image> <shell>
```

Flag	Nom complet	Fonction
<code>-i</code>	<code>--interactive</code>	Garde l'entrée standard (STDIN) ouverte, vous permettant de taper des commandes
<code>-t</code>	<code>--tty</code>	Alloue un pseudo-terminal pour une expérience de shell interactive



Important : Les deux flags sont généralement utilisés ensemble (**-it**) pour une session interactive complète.

Interactif avec docker run : exemple

Méthode 1 : Démarrer un Shell Interactif avec **docker run**

- Exemple pratique

```
sudo docker run -it ubuntu bash
```

Ce qui se passe à l'exécution de cette commande :

1. Docker télécharge l'image Ubuntu (si elle n'est pas déjà présente)
2. Un nouveau conteneur est créé à partir de cette image
3. Le shell bash est lancé à l'intérieur du conteneur
4. Vous obtenez un prompt [@Linux] interactif

Interactif avec docker run : autres shells Linux

Méthode 1 : Démarrer un Shell Interactif avec **docker run**

Utiliser sh au lieu de bash

```
docker run -it ubuntu sh
```

Utiliser zsh (si disponible dans l'image)

```
docker run -it ubuntu zsh
```

Lancer directement une commande Python

```
docker run -it python:3.9 python
```

Interactif avec docker exec : principe

Méthode 2 : Démarrer un Shell Interactif avec **docker exec**

Principe

- La commande **docker exec** permet d'exécuter des commandes dans un conteneur déjà en cours d'exécution.
- C'est la méthode la plus courante pour le débogage, la vérification des logs ou l'installation de paquets à la volée.

Interactif avec docker exec

Méthode 2 : Démarrer un Shell Interactif avec **docker exec**

- **Étape 1 : Lister les conteneurs**

- Trouvez l'ID ou le nom de votre conteneur en cours d'exécution :

- `sudo docker ps` *# ou* `sudo docker container ls`

- Exemple de sortie

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	NAMES
d64b00529582	ubuntu	"bash"	2 minutes ago	Up 2 minutes	mon_conteneur

Interactif avec docker exec

Méthode 2 : Démarrer un Shell Interactif avec **docker exec**

- **Étape 2 : Démarrer le conteneur (si nécessaire)**

- Si le conteneur est arrêté, démarrer-le :

```
sudo docker container start d64b00529582
```

Ou en mentionnant le nom :

```
sudo docker start mon_conteneur
```

Interactif avec docker exec

Méthode 2 : Démarrer un Shell Interactif avec **docker exec**

- **Étape 3 : Exécuter une commande avec exec**

```
sudo docker exec -it d64b00529582 bash
```

Syntaxe général :

```
docker exec [OPTIONS] <container_id_ou_nom> <commande>
```

Interactif avec docker exec

`docker exec` [OPTIONS] <container_id_ou_nom> <commande>

Option	Description	Exemple
<code>-i</code>	Mode interactif	<code>docker exec -i conteneur cat > fichier.txt</code>
<code>-t</code>	Alloue un terminal	<code>docker exec -t conteneur ls</code>
<code>-it</code>	Combinaison pour shell interactif	<code>docker exec -it conteneur bash</code>
<code>-d</code>	Exécution en arrière-plan (détaché)	<code>docker exec -d conteneur script.sh</code>
<code>-e</code>	Définir une variable d'environnement	<code>docker exec -e VAR=valeur conteneur env</code>
<code>-u</code>	Spécifier l'utilisateur	<code>docker exec -u root conteneur whoami</code>
<code>-w</code>	Définir le répertoire de travail	<code>docker exec -w /app conteneur pwd</code>

Interactif avec docker exec : Exemples pratiques (1/3)

Méthode 2 : Démarrer un Shell Interactif avec **docker exec**

- **Exemple 1 : Lister les fichiers**

```
docker exec mon_conteneur ls -la /var/log
```

Exemple 2: Vérifier les processus

```
docker exec mon_conteneur ps aux
```

Exemple 3: Installer un paquet

```
docker exec -it mon_conteneur apt-get update
```

```
docker exec -it mon_conteneur apt-get install -y curl
```


Interactif avec docker exec : Exemples pratiques (2/3)

Méthode 2 : Démarrer un Shell Interactif avec **docker exec**

- **Exemple 4 : Exécuter un script**

```
docker exec mon_conteneur python /app/script.py
```

Exemple 5 : Exécuter en arrière plan

```
docker exec -d mon_conteneur /app/background-task.sh
```

Interactif avec docker exec : Exemples pratiques (3/3)

Méthode 2 : Démarrer un Shell Interactif avec **docker exec**

- **Exemple 6 : Ouvrir un shell interactif**
- L'une des méthodes les plus utilisées pour explorer un conteneur

Avec bash

```
docker exec -it mon_conteneur bash
```

Avec sh (si bash n'est pas disponible)

```
docker exec -it mon_conteneur sh
```

Avec un utilisateur spécifique

```
docker exec -it -u root mon_conteneur bash
```

